

Small_SoC: Design, Verification and FPGA Implementation of a From-Scratch RV32I Microcontroller System-on-Chip with an AXI-Lite Fabric and a JTAG Debug Port

Krishna Gorai

Abstract—This report describes Small_SoC, a complete microcontroller-class system-on-chip written from scratch in synthesisable Verilog-2001 without any vendor intellectual property. The system consists of a multicycle RV32I processor core with the Zicsr extension and machine-mode trap handling, a hand-written AMBA AXI4-Lite interconnect, a 32 KB block-RAM program memory, general-purpose I/O, a UART, a timer with an interrupt line, and a JTAG debug port that can halt the core, access every bus address and load a program without rebuilding the bitstream. A C tool-chain (linker script, start-up code, trap vector and board-support library) targets the SoC with an unmodified GNU compiler. Correctness is established at four levels: the core passes all 41 applicable programs of the official riscv-tests rv32ui suite; two application programs run as self-checking simulations; the debug port is exercised end-to-end by a JTAG bus-functional model; and the placed-and-routed netlist is simulated at gate level with the real board clock. On a Xilinx Zynq UltraScale+ XCZU7EV (ZCU104 board) the SoC occupies 1 710 look-up tables, 985 flip-flops and 8 block RAMs, meets timing at 100 MHz with 5.3 ns of slack, and the debug port adds about 190 look-up tables. The report gives the architecture, the working mechanism of every block, the verification methodology and the implementation results, and records the design decisions and the problems met along the way.

Index Terms—RISC-V, system-on-chip, AXI4-Lite, JTAG, boundary scan, FPGA, Verilog, processor design, hardware verification.

I. INTRODUCTION

A system-on-chip (SoC) is a processor together with the memory, the interconnect and the peripherals it needs to run a program and to talk to the outside world. Commercial SoCs are assembled from licensed blocks and vendor generators, which hides the mechanisms a designer must understand to build, verify and debug one. Small_SoC is the opposite exercise: every block, from the instruction decoder to the JTAG test-access port, is written by hand in plain Verilog-2001, so that every line can be explained and

every behaviour can be traced to the RTL that produces it.

The design implements the 32-bit RISC-V base integer instruction set (RV32I) [1] together with the control-and-status-register extension (Zicsr) and the machine-mode subset of the privileged architecture [2] that is needed for traps and interrupts. The core talks to its memory and peripherals through the AMBA AXI4-Lite protocol [3], the lightweight subset of AXI4 used for register-mapped devices. A debug port, reached through the FPGA's own JTAG chain [4], lets a host stop the core, inspect and modify memory and peripheral registers, and load programs. The SoC is implemented on the Xilinx Zynq UltraScale+ ZCU104 evaluation board [6] and runs C programs produced by the unmodified GNU tool-chain.

The report is organised as follows. Section II states the goals and the constraints they impose. Section III recalls the standards the design implements. Section IV presents the system architecture and the memory map. Sections V to VIII describe the working mechanism of the core, the bus, the peripherals and the debug port. Section IX covers the software stack, Section X the verification methodology, and Section XI the FPGA implementation and its results. Section XII records the problems encountered and how they were resolved, and Section XIII concludes with future work.

II. MOTIVATION AND OBJECTIVES

The project has one primary goal and four derived requirements.

Goal. Produce a small but complete SoC that a single designer can defend in full: no block may be a black box, and every claim about it must be backed by a reproducible test.

Correctness before performance. The core is a multicycle machine rather than a pipeline. A pipeline would raise instructions per cycle but would add hazards, forwarding and flushing logic whose verification would dominate the project. A multicycle controller has one instruction in flight at a time; its correctness can be argued state by state and confirmed with the standard instruction-set tests.

K. Gorai is with the Indian Institute of Technology Mandi, India (e-mail: kg05112001@gmail.com). Project repository: <https://github.com/Krishna-Gorai/small-soc-riscv>.

Standard interfaces. The bus is AXI4-Lite and the debug port is reached through IEEE 1149.1 JTAG, so that the blocks can be understood, and reused, by anyone who knows those standards, and so that the debug port works with the board's existing USB-JTAG cable rather than needing a dedicated header.

Real software. The SoC must run C code compiled by an unmodified GNU compiler, which requires a linker script, start-up code and a trap vector that match the hardware exactly.

Evidence at every level. Register-transfer-level (RTL) simulation alone does not prove that a design works on silicon. The project therefore also checks the placed-and-routed netlist by simulation and reports the implementation numbers from the vendor tools.

III. BACKGROUND

A. RV32I, Zicsr and Machine Mode

RV32I [1] has 32 general-purpose registers of 32 bits ($x0$ is hard-wired to zero), a 32-bit program counter, and 40 instructions in six encoding formats (R, I, S, B, U, J). All instructions are 32 bits wide and word-aligned. Loads and stores access bytes, half-words and words; branches compare two registers; `jal/jalr` implement calls and indirect jumps; `lui/auipc` build 32-bit constants and PC-relative addresses.

Zicsr adds six instructions (`csrrw`, `csrrs`, `csrrc` and their immediate forms) that read and modify the control-and-status registers (CSRs), a separate 4096-entry address space. The privileged architecture [2] defines, for machine mode (M-mode), the CSRs that control traps: `mstatus` (global interrupt enable MIE and its saved copy MPIE), `mie/mip` (per-source interrupt enable and pending bits), `mtvec` (trap vector), `mepc` (return address), `mcause` (trap cause) and `mscratch`. A trap saves the PC in `mepc`, records the cause, disables interrupts and jumps to `mtvec`; the `mret` instruction reverses this. `ecall` and `ebreak` raise synchronous traps from software.

B. AMBA AXI4-Lite

AXI4-Lite [3] is a memory-mapped bus with five independent channels: read address (AR), read data (R), write address (AW), write data (W) and write response (B). Each channel carries a `VALID` from its source and a `READY` from its sink; a transfer takes place on the clock edge where both are high. A source must not withdraw `VALID` once asserted until the transfer completes, and must not make `VALID` depend on `READY`. All transfers are single 32-bit words; writes carry a 4-bit byte strobe (`WSTRB`). Because the channels are independent, AW and W may arrive in either order and a slave must accept both before responding on B.

C. IEEE 1149.1 JTAG and Xilinx BSCANE2

The JTAG test-access port (TAP) [4] is a four-wire serial interface (TCK, TMS, TDI, TDO) driven by a 16-state

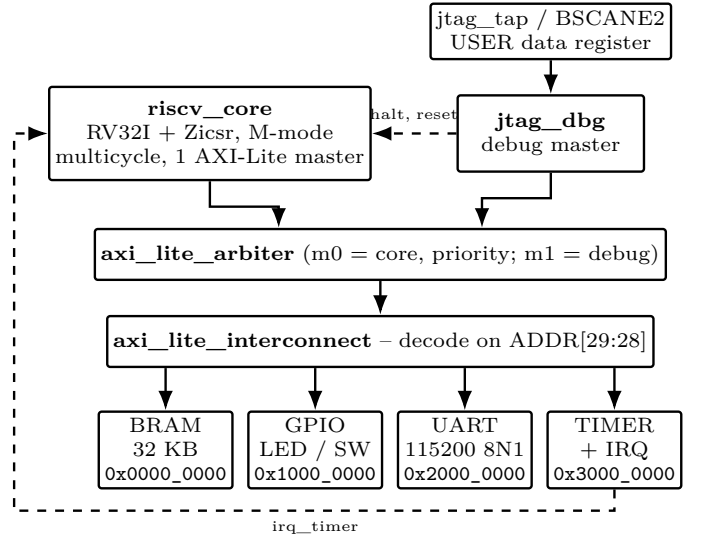


Fig. 1. Top level of Small_SoC. Solid arrows are AXI4-Lite master-to-slave connections; dashed arrows are side-band control.

controller. TMS steers the controller between the instruction-register (IR) and data-register (DR) scan paths; in the *Shift* states one bit enters on TDI and one leaves on TDO per TCK; the *Capture* state loads a register before shifting and the *Update* state commits it afterwards. Every device has a *BYPASS* and an *IDCODE* instruction; vendors add *user* instructions that connect the DR path to user logic. On Xilinx UltraScale+ devices the BSCANE2 primitive [5] exposes one such user register: it presents *SEL*, *CAPTURE*, *SHIFT*, *UPDATE*, *TCK*, *TDI* to fabric logic and takes TDO back, so a design can hang its own serial register off the device's JTAG chain without any package pins.

IV. SYSTEM ARCHITECTURE

Fig. 1 shows the SoC. One RISC-V core and one JTAG debug master share a single AXI4-Lite fabric through a two-master arbiter; the interconnect behind it decodes four slaves. The timer's interrupt line goes back to the core; the debug port's halt and reset lines go to the core as well. The whole design runs on one clock (`clk`, 100 MHz on the board) and one active-low asynchronous reset (`rst_n`), except for the debug register, which is clocked by JTAG TCK and crosses into the `clk` domain through synchronisers (Section VIII).

A. Memory Map

The interconnect decodes address bits [29:28], which gives each slave a 256 MB window; each slave decodes bits [3:2] for its own registers and ignores the rest. Table I lists the map. Addresses inside a window that a slave does not implement alias onto its registers; there is no decode-error response, a simplification that is acceptable in a closed SoC where all software is trusted.

B. Clocking and Reset

Inside `soc_top` the board reset is passed through a two-flip-flop synchroniser whose output, `rst_n`, is the

TABLE I
MEMORY MAP AND REGISTER DEFINITIONS

Base	Block	Registers (offset: name, access)
0x0000_0000	BRAM	32 KB, byte-writable; code, data, stack; initialised from <code>program.hex</code> in the bitstream
0x1000_0000	GPIO	+0 OUT (RW, LEDs); +4 IN (RO, switches and buttons)
0x2000_0000	UART	+0 TX (W: send byte; R: last byte); +4 RX (R, clears <code>rx_valid</code>); +8 STATUS {bit1 <code>rx_valid</code> , bit0 <code>tx_busy</code> }
0x3000_0000	Timer	+0 COUNT (RO); +4 COMPARE (RW); +8 CTRL {bit1 <code>irq_en</code> , bit0 <code>enable</code> }; +C STATUS {bit0 <code>pending</code> , write 1 to clear}

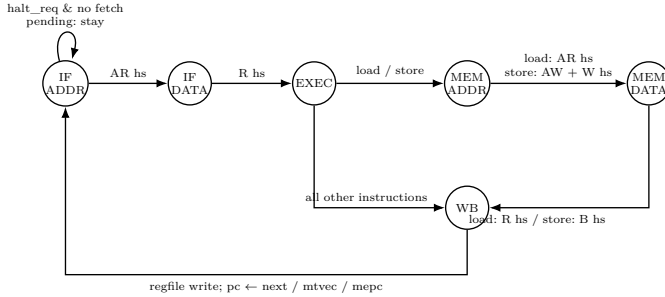


Fig. 2. Core controller. “hs” denotes an AXI handshake ($\text{VALID} \wedge \text{READY}$). A debug halt request is honoured in `IF_ADDR` before a fetch is issued.

asynchronous reset of every block. The core receives `rst_n` ANDed with the inverse of the debug port’s core-reset request, so a debugger can reset the processor alone while the peripherals, the memory contents and the debug logic keep their state. The ZCU104 wrapper (Section XI) derives the 100 MHz clock from the board’s 300 MHz differential oscillator with a single `BUFGCE_DIV` primitive dividing by three; there is no MMCM, so no lock signal has to be handled and no clocking IP has to be regenerated.

V. PROCESSOR CORE

A. Control: a Six-State Multicycle Machine

The core executes one instruction at a time under the controller of Fig. 2. `IF_ADDR` issues the fetch address on the AR channel; `IF_DATA` waits for the instruction on the R channel and latches it into `instr_reg`; `EXEC` evaluates the instruction; loads and stores pass through `MEM_ADDR` and `MEM_DATA` for the data-side bus transaction; `WB` writes the destination register, updates the program counter and resolves traps. Table II gives the cost per instruction class with the zero-wait-state slaves of this SoC; the run-time counters `mcycle` and `minstret` confirm an average of about 5.6 cycles per instruction on the demonstration programs.

B. Datapath

All decode is combinational from `instr_reg`: the opcode, register indices and the five immediate formats are wires. The ALU is a 32-bit combinational block with eleven

TABLE II
CYCLES PER INSTRUCTION CLASS

Class	Cycles	States visited
ALU (R/I), LUI, AUIPC, branch, JAL, JALR, CSR, system	5	IF_ADDR(2), IF_DATA, EXEC, WB
Load, store	7	+ MEM_ADDR, MEM_DATA

operations (add, subtract, and, or, xor, three shifts, two compares and a pass-through used by `lui`); its operands are selected by instruction class (pc for `auipc/jal`, zero for `lui`, `rs1` otherwise; `rs2` or the appropriate immediate). A separate comparator evaluates the six branch conditions. The next PC is chosen among `pc+4`, the branch target, the `jal` target and the `jalr` target (with bit 0 cleared as the specification requires).

The register file is `regs[1:31]`; `x0` is a multiplexer that returns zero rather than storage. Reads are combinational; the write port is registered (`rd_we`, `rd_waddr`, `rd_wdata`) so the write lands in the cycle after `WB`, which is always before the next instruction reaches `EXEC`. Vivado maps it to 40 LUT-RAMs.

C. Loads and Stores

The core issues only word-aligned bus transactions. For a load, the word returned on the R channel is steered and extended according to `funct3` (byte, half-word or word; signed or zero-extended) and the two low address bits latched in `mem_addr_reg`. For a store, the data is placed in the correct byte lanes and `WSTRB` is set to the matching pattern, so a `sb` to address `0x...3` drives `WDATA[31:24]` with `WSTRB=4'b1000`. Naturally aligned accesses are therefore correct for all five load and three store instructions; misaligned accesses are not supported and raise no exception, which is why the `ma_data` test is excluded from the regression.

D. Bus Master Behaviour

The core’s AXI outputs are registered. In `IF_ADDR` it raises `ARVALID` with the PC on `ARADDR` and holds both until `ARREADY`; it then raises `RREADY` and holds `ARADDR` stable until the data arrives. For a store it raises `AWVALID` and `WVALID` together and tracks their acceptance separately (the `aw_done/w_done` flags), asserting `BREADY` only when both have completed. Holding the address stable for the whole transaction is what lets the interconnect route the response channels by address alone.

E. Control and Status Registers

Table III lists the implemented CSRs. `csr_rdata` is a combinational read multiplexer; the write value is computed from the operation (`csrrw` writes the operand, `csrrs` ORs it in, `csrrc` clears it) using either `rs1` or the 5-bit immediate. A `csrrs/csrrc` with `rs1=x0` is a pure read, as the specification requires. `mcycle` increments every clock;

TABLE III
IMPLEMENTED CONTROL AND STATUS REGISTERS

CSR	Address	Implemented fields
mstatus	0x300	MIE (bit 3), MPIE (bit 7)
misa	0x301	read-only 0x4000_0100 (RV32I)
mie	0x304	MTIE (bit 7)
mtvec	0x305	base, direct mode
mscratch	0x340	32 bits
mepc	0x341	bits [31:2]
mcause	0x342	32 bits
mip	0x344	MTIP (bit 7) = timer IRQ input, read-only
mcycle[h], cycle[h]	0xB00/B80, 0xC00/C80	64-bit clock counter
minstret[h], instret[h]	0xB02/B82, 0xC02/C82	64-bit retired-instruction counter
mvendorid, marchid, mimpid, mhartid, mtval	0xF11-F14, 0x343	read as 0

TABLE IV
TRAP HANDLING IN WB, IN PRIORITY ORDER

Event	mcause	mepc	Next PC
ecall	11	PC of ecall	mtvec
ebreak	3	PC of ebreak	mtvec
mret	—	—	mepc
timer interrupt	0x8000_0007	next PC	mtvec
none	—	—	next PC

minstret increments in WB. Unknown CSRs read as zero and ignore writes.

F. Traps and Interrupts

Traps are resolved in WB in the priority order of Table IV. A synchronous trap or an interrupt copies MIE into MPIE, clears MIE, records the cause and jumps to mtvec; mret restores MIE from MPIE, sets MPIE and returns to mepc. An interrupt is taken only when mstatus.MIE, mie.MTIE and the level input irq_timer are all set; because the check is made in WB, an interrupt never splits an instruction and mepc receives the address of the instruction that would have executed next, which is exactly what mret must return to. The fence, fence.i and wfi instructions are accepted and execute as no-ops: there are no caches or write buffers, and the only other bus master is serialised with the core by the arbiter, so no ordering action is needed.

G. Debug Hook

The debug port presents dbg_halt_req. It is honoured in IF_ADDR before the fetch is issued: while the request is high and no fetch is pending the controller stays in IF_ADDR and reports dbg_halted. A halted core therefore has no bus transaction in flight, which is what makes it safe for the debug master to take the bus and for the debugger to reset the core.

VI. INTERCONNECT AND ARBITER

A. Interconnect

axi_lite_interconnect is purely combinational. Read channels are steered by ARADDR[29:28] and write channels

by AWADDR[29:28]: the per-slave VALID and READY signals are gated with the decoded select, and the response signals (RDATA, RRESP, RVALID, BRESP, BVALID) are multiplexed back with the same select. No transaction identifier is stored because the master never has more than one read and one write outstanding and holds the address until the response is accepted, so the select is valid for the whole transaction.

B. Arbiter

axi_lite_arbiter places the core (m0) and the debug master (m1) in front of the interconnect. A one-bit registered grant selects whose signals reach the slave port. m1 is granted only when it has a request and m0 has nothing in flight (no VALID asserted and no response awaited); it keeps the grant until its own transaction completes. While m1 holds the bus, m0's VALIDs are masked from the slave and its READYs are held low, so the core waits with its request asserted, which the AXI rules allow. No transaction is ever truncated, and the core sees only a stall of a few cycles per debug access. Because the core's request and the grant are both registered, a fetch raised in the same cycle that the grant switches is simply held until the grant returns.

VII. PERIPHERALS

All four slaves share one handshake structure per direction: an idle state in which READY is high, capture on the handshake, and a response state in which VALID is held until accepted. Writes are accepted only when AW and W are presented in the same cycle, which both masters always do.

A. Block RAM Controller

The memory is an array of 8192 32-bit words with byte write enables. The array lives in its own clocked block without a reset, which is the coding style Vivado requires to infer block RAM; the read data register is loaded on the AR handshake and returned one cycle later. The array is initialised with \$readmemh("program.hex"), kept outside any translate_off region so that synthesis reads the same file and the program becomes part of the bitstream (8 RAMB36E2 primitives with their INIT strings).

B. GPIO

An output register with byte strobes drives the LEDs; the input port is sampled directly on read. The board wrapper feeds the four DIP switches and four push buttons to the input.

C. UART

The UART is 8N1 with a parameter CLKS_PER_BIT (868 at 100 MHz for 115200 baud; the testbenches use 20 to shorten simulations). The transmitter is a four-state machine (idle, start, eight data bits, stop) with a one-byte holding register and a tx_busy flag; the receiver synchronises the asynchronous line with two flip-flops, waits

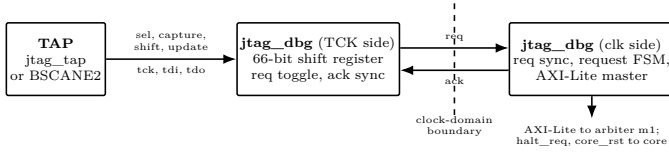


Fig. 3. Structure of the debug port. The two right-hand blocks form `jtag_dbg`; the request and acknowledge toggles are the only signals that cross between the TCK and `clk` domains.

half a bit period after the start edge to confirm it, then samples each bit at its centre, and sets `rx_valid` after the stop bit. Reading the RX register clears the flag. There are no FIFOs: a byte arriving while the previous one is unread is lost, and the software must poll `STATUS` at least once per byte time.

D. Timer

A 32-bit counter runs while `CTRL.enable` is set and returns to zero after reaching `COMPARE`, so the period is `COMPARE+1` cycles. Each wrap sets `STATUS.pending`; the interrupt output is `CTRL.irq_en ^ pending`. The pending flag is cleared by writing 1 to `STATUS` (write-1-to-clear), with the set taking priority if both happen in the same cycle, so no wrap is ever lost.

VIII. JTAG DEBUG PORT

A. Purpose and Structure

The debug port gives a host three capabilities: halt, resume and reset the core; read and write any address on the SoC bus; and, as a consequence, load a program into the BRAM and start it without rebuilding the bitstream. It is built from two blocks (Fig. 3). `jtag_tap` is an IEEE 1149.1 TAP with a 4-bit instruction register (`IDCODE`, `BYPASS`, `USER`); it is used in simulation and could drive a discrete JTAG header. `jtag_dbg` is the user data register and the bus master behind it. The interface between them is the seven-signal set that Xilinx’s `BSCANE2` primitive presents (`SEL`, `CAPTURE`, `SHIFT`, `UPDATE`, `TCK`, `TDI`, `TDO`), so on the FPGA the same `jtag_dbg` hangs off the device’s own JTAG chain (`USER1` instruction) and is reached through the board’s USB-JTAG cable with no additional pins.

B. Data Register Protocol

The debug data register is 66 bits and is shifted least-significant bit first (Table V). The host shifts in an operation, a data word and an address; the request is issued on *Update-DR*. Each subsequent scan captures the response: bit 0 is `busy`, bits [33:2] carry the data returned by the last completed read, and bits [36:34] report the core status. The host polls with `op=nop` until `busy` clears. A load therefore costs two to three 66-bit scans per word; at a 10 MHz TCK the 1.5 KB `hello` program loads in about 8 ms.

TABLE V
DEBUG DATA REGISTER (66 BITS, LSB FIRST)

Bits	Shift in (host → SoC)	Shift out (SoC → host)
[1:0]	op: 0 nop, 1 read, 2 write, 3 control	bit 0 busy, bit 1 zero
[33:2]	data; for control: bit 0 halt, bit 1 core reset	last read data
[34]		halted
[35]	address [31:0] (bits	halt request asserted
[36]	[65:34])	core reset asserted
[65:37]		zero

C. Clock-Domain Crossing

TCK and `clk` are unrelated. On *Update-DR* the TCK-domain logic copies the request fields into holding registers and inverts a request toggle. A three-flip-flop synchroniser in the `clk` domain detects the toggle edge (the first two flops synchronise, the third provides the edge), at which point the holding registers have been stable for at least two `clk` periods and are read safely. When the transaction completes the `clk` domain inverts an acknowledge toggle, which is synchronised back into the TCK domain by two flops; `busy` is the exclusive-or of the two toggles. The response payload (`rdata`, `halted`) is only sampled by the TCK side after `busy` has cleared, so it too is stable when read. Both synchronisers carry the `ASYNC_REG` attribute; in the constraints TCK is declared as a clock (30 MHz, above the hardware manager’s 15 MHz default) and placed in an asynchronous clock group with the SoC clock.

D. Bus Master and Core Control

The `clk`-domain state machine performs one AXI-Lite read or write per request with the same registered-output discipline as the core, or, for a control request, updates `dbg_halt_req` and `dbg_core_rst`. The usual sequence to load and run a program is: halt; write the image word-by-word; optionally read it back; assert and release core reset with halt still asserted (the core restarts at address 0 but does not fetch); then release halt.

E. Host Side

`fpga/jtag_host.tcl` drives the port from the Vivado hardware manager with the raw `scan_ir_hw_jtag` and `scan_dr_hw_jtag` commands. The instruction-register length (12 bits) and the `USER1` opcode (0x902) are taken from the XCZU7EV boundary-scan description file shipped with Vivado. The script provides `dbg_halt`, `dbg_run`, `dbg_reset`, `dbg_peek`, `dbg_poke` and `dbg_load`.

IX. SOFTWARE STACK

The SoC runs C compiled by the unmodified xPack GNU tool-chain (`riscv-none-elf-gcc 15.2`) with `-march=rv32i_zicsr_zifencei -mabi=ilp32`. Four small pieces of code bridge the compiler and the hardware.

Linker script. One 32 KB region at address 0 holds `.text` (with the start-up section placed first, at the reset

vector), `.rodata`, `.data` and `.bss`; the stack grows down from the top. Because the whole image is loaded into the BRAM, `.data` needs no copy at start-up. An assertion fails the link if code, data and a 2 KB stack do not fit.

Start-up code. `crt0.S` sets the global and stack pointers, writes the trap vector into `mtvec`, zeroes `.bss`, calls `main` and parks in a loop if it returns.

Trap vector. `trap.S` saves the sixteen caller-saved registers on the stack, calls the C function `trap_handler(mcause, mepc)`, writes its return value back to `mepc` (`mepc+4` for `ecall` and `ebreak`, unchanged for an interrupt), restores the registers and executes `mret`. A weak default handler reports the cause on the UART and halts; programs override it.

Board-support library. `soc.h` defines the memory map, register bits and CSR access macros; `uart.c` and `timer.c` provide polled character output and input, decimal and hexadecimal printing, and delays.

Build flow. A shared `rules.mk` gives every program three targets: `make` builds the ELF, a raw binary, the `$readmemh` image and a disassembly listing; `make sim` rebuilds with `-DSIM_BUILD` (short delays, self-terminating) and runs it in the simulator with the UART output shown on the console; `make install` copies the image into `rtl/program.hex` for the next bitstream. The ELF-to-image conversion is a 15-line Python script.

Programs. `hello` (1477 bytes) prints a banner and the memory map, walks a pattern across the LEDs paced by the timer, reports the switches whenever they change and echoes received characters. `irq_demo` (1325 bytes) issues an `ecall`, then enables the timer interrupt and prints the tick count, `mcycle` and `minstret` from the main loop while the interrupt service routine acknowledges the timer and toggles an LED. Built with `-DSIM_BUILD`, each program writes a completion code to the GPIO output that the testbench recognises, so both double as regression tests.

X. VERIFICATION

Verification proceeds from the instruction set upward to the routed netlist. All simulations use the Vivado simulator (`xsim`); every step is scripted and reproducible from the repository.

A. Instruction-Set Regression

The official `riscv-tests rv32ui` suite [7] is vendored. Its stock environment relies on CSRs and trap paths (`tohost`, misaligned traps) that a minimal core does not have, so a 40-line `riscv_test.h` was written that reports PASS by writing `0x0000600D` to the GPIO output and FAIL by writing `0xBAD00000|test number`, together with a linker script that places each test at address 0. The runner `run_isa.py` compiles each program, converts it to an image, simulates it under the generic testbench `tb_isa.v` and tabulates the results; the whole suite takes about two minutes. All 41 applicable tests pass (Table VI); `ma_data` is excluded because it requires misaligned-access traps. The harness was checked for sensitivity by replacing

TABLE VI
RISCV-TESTS RV32UI RESULTS (CYCLES TO PASS, RTL SIMULATION)

add	2147	bltu	1402	lui	147	sltiu	1007
addi	1032	bne	1277	lw	1189	sltu	2117
and	2247	fence_i	1250	or	2262	sra	2382
andi	812	jal	97	ori	847	srai	1102
auipc	117	jalr	397	sb	2119	srl	2352
beq	1277	lb	1039	sh	2384	srlr	1072
bge	1367	lbu	1039	simple	27	st_ld	2286
bgeu	1492	ld_st	5795	sll	2287	sub	2107
blt	1277	lh	1119	slli	1027	sw	2421
		lhu	1164	slt	2117	xor	2257
				slti	1007	xori	857

41 / 41 PASS

the ALU's subtraction with addition: `sub` then fails at its third sub-test while every other test is unaffected.

B. Generic Program Testbench

`tb_isa.v` loads any image into the BRAM array, decodes the SoC's UART transmit line and prints it on the console, optionally injects characters into the receive line once the transmitter has been quiet for 50 μ s (so the stimulus lands after any banner), and stops on the PASS/FAIL code or a watchdog. The same testbench runs the ISA suite and the application programs.

C. Application Programs

`hello` passes with the full banner, the switch report, the echo of the injected characters and the completion code; `irq_demo` passes with the `ecall` acknowledged and three timer interrupts serviced, each tick printed with the cycle and instruction counters.

D. Debug Port

`tb_jtag.v` contains a JTAG bus-functional model that drives real TMS/TDI sequences into `jtag_tap`. With the BRAM initially holding only `jal x0,0`, it reads `IDCODE`, halts the core and checks the halted flag, writes the 382-word `hello` image through the debug master and reads every word back, resets and releases the core, then, while the program runs and prints, reads `GPIO_IN` and `TIMER_COUNT` over JTAG (which also exercises the arbiter under contention), and finally waits for the program's completion code. The test passes in 1.43 M clock cycles of simulation.

E. Post-Implementation Simulation

After place and route, `write_verilog -mode funcsim` produces the gate-level netlist of `zcu104_top`, including the block RAMs with their initialisation strings and the `BSCANE2`. A board-level testbench drives the real 300 MHz differential clock, leaves the reset button released (so the wrapper's 128-cycle self-reset is what starts the SoC), decodes the UART at 115200 baud and passes when the banner begins to appear. The routed netlist prints `Hello` at $t = 610 \mu$ s; the simulation takes about 35 s.

TABLE VII
ZCU104 PIN ASSIGNMENT

Signal	Package pins	I/O standard
clk300_p/n	AH18 / AH17	DIFF_SSTL12
cpu_reset (active high)	M11	LVC MOS33
dip[3:0]	E4, D4, F5, F4	LVC MOS33
pb[3:0]	B4, C4, B3, C3	LVC MOS33
led[3:0]	D5, D6, A5, B5	LVC MOS33
uart_tx / uart_rx (CP2108 PL channel)	C19 / A20	LVC MOS18
JTAG	device TAP via BSCANE2	–

XI. FPGA IMPLEMENTATION

A. Board Wrapper and Constraints

`zcu104_top` adds to `soc_top` an `IBUFDS` for the 300 MHz DDR4-bank oscillator, a `BUFGCE_DIV` dividing by three, a two-flop synchroniser and 128-cycle stretcher on the `CPU_RESET` button, the `BSCANE2` with a `BUFG` on its `TCK`, and the pin mapping of Table VII. Pin locations and I/O standards are taken from the ZCU104 board file shipped with Vivado. The constraints define the 300 MHz input clock (the 100 MHz clock is derived automatically), the 30 MHz `TCK` clock in an asynchronous group, and false paths on the switch, button, LED and UART pins, which have no timing requirement.

B. Flow

A non-project Tcl script runs `synth_design`, `opt_design`, `place_design`, `phys_opt_design`, `route_design` and `write_bitstream` with default directives and writes the utilisation, timing, power and DRC reports. A second script regenerates the Vivado GUI project from the same sources for interactive work. The flow takes about 25 minutes on an 8 GB laptop, dominated by placement.

C. Results

Table VIII gives the utilisation by block and Table IX the timing summary. All constraints are met. The worst setup path on the 100 MHz clock leaves 5.3 ns of the 10 ns period unused, so the design would close at about 200 MHz without modification; 100 MHz was kept so that the UART divisor and the timer values remain round numbers. The critical path (4.55 ns, 9 logic levels, 77 % routing) runs from the instruction register through the register-file read (`funct3` selects the branch condition, `rs1/rs2` address the LUT-RAM), the branch comparator and the next-PC multiplexer to `pc_reg`, as expected for a multicycle core that resolves branches combinatorially in the same state that decodes them.

The whole SoC uses 0.74 % of the device’s look-up tables and 2.6 % of its block RAM. The JTAG debug port (debug register, synchronisers, bus master, arbiter and the core’s halt hook) costs about 190 look-up tables and 230 flip-flops, about 11 % of the SoC, and left the SoC clock’s slack essentially unchanged (from +5.36 to +5.28 ns). The

TABLE VIII
UTILISATION ON THE XCZU7EV AFTER PLACE AND ROUTE

Instance	Module	LUTs	LUTRAM	FFs	RAMB36	DSP
<code>zcu104_top</code>	wrapper + SoC	1 717	40	995	8	0
<code>u_soc</code>	<code>soc_top</code>	1 710	40	985	8	0
<code>u_core</code>	<code>riscv_core</code>	1 400	40	488	0	0
<code>u_dbg</code>	<code>jtag_dbg</code>	164	0	228	0	0
<code>u_uart</code>	UART + tx + rx	73	0	87	0	0
<code>u_timer</code>	timer	66	0	103	0	0
<code>u_gpio</code>	GPIO	1	0	72	0	0
–	BRAM ctrl, arbiter, xbar	≈ 6	0	≈ 7	8	0

TABLE IX
TIMING SUMMARY (SETUP/HOLD/PULSE-WIDTH SLACK)

Clock	Period (ns)	Endpoints	WNS (ns)	WHS (ns)	WPWS (ns)
clk (SoC)	10.0	2 370	+5.275	+0.003	+2.791
jtag_tck	33.3	165	+31.228	+0.017	+16.391
clk300 (input)	3.33	1	–	–	+2.262

vectorless power estimate is 0.631 W, of which only 0.017 W is dynamic: the design is far too small to move the device’s static floor.

XII. PROBLEMS ENCOUNTERED AND DESIGN LESSONS

The following problems arose during the project; each changed either the design or the methodology.

A hand-assembled instruction. The first test program stored its result to address 0xF instead of the GPIO register and the LED never changed. A cycle-by-cycle trace showed the core computing exactly what it was told: the store had been hand-encoded with `rs1=x3` instead of `x4`. The lesson is that programs must come from an assembler and that a regression must be independent of the program under test; both led to the `riscv-tests` flow.

Simulation-only initialisation. The BRAM’s `$readmemh` was originally inside a `translate_off` region, so the bitstream would have contained an empty program memory. Making the initialisation synthesisable, and moving the memory array into a reset-free clocked block so that block RAM is inferred rather than 8 192 words of distributed RAM, fixed both the function and the area.

Timescale and reset discipline. The design files carried no `'timescale`; the GUI hid the resulting elaboration error with `-relax`. Twenty-three registers were assigned in asynchronous-reset blocks without a reset value and produced synthesis warnings. Both were corrected, and the standalone simulation flow deliberately elaborates without `-relax` so that such issues are visible.

Vendor batch wrappers. The Vivado `.bat` wrappers on Windows split arguments at “=”, so simulator plus-arguments cannot carry a path. The testbenches therefore read their image from a fixed file name and their watchdog value from a small text file, which is also simpler to script.

Single-byte UART receiver. Characters injected while the program was still printing its banner were overwritten in the receiver’s one-byte register. Rather than hide this with a magic delay, the testbench waits for the transmit line to be idle before injecting, and the limitation is documented for software.

TDO sampling race. The first JTAG testbench sampled TDO in the same time step as the falling TCK edge that updates it and read every word shifted by one bit. Sampling a quarter period after the edge removed the race; the RTL was unchanged.

Unconstrained JTAG clock. The first build with BSCANE2 had no clock defined on its TCK, so the debug register’s paths would have been unconstrained, silently. A clock constraint and an asynchronous clock group were added before the results were recorded.

XIII. CONCLUSION AND FUTURE WORK

Small_SoC demonstrates a complete, from-scratch SoC whose every block is understood and tested: an RV32I core with machine-mode traps and interrupts that passes the official instruction-set suite, an AXI4-Lite fabric with two masters, four peripherals, a JTAG debug port that loads and controls programs through the FPGA’s own test-access port, and a C tool-chain. The placed-and-routed design occupies 1 710 LUTs and 8 block RAMs on the XCZU7EV, meets timing at 100 MHz with 5.3 ns to spare, and its netlist boots and prints in gate-level simulation.

Three extensions are natural next steps. An illegal-instruction exception and misaligned-access traps would complete the M-mode exception model and allow the remaining `riscv-tests` programs to run. A two- or three-stage pipeline would raise the throughput from 5–7 cycles per instruction towards one, at the cost of hazard logic that the existing regression is well placed to verify. And extending the debug port to the RISC-V External Debug Specification (a debug module with abstract commands and a debug-mode CSR set) would let `gdb` attach through OpenOCD, building on the halt, reset and memory-access mechanisms already in place.

REFERENCES

- [1] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, RISC-V Foundation, Dec. 2019.
- [2] A. Waterman, K. Asanović, and J. Hauser, Eds., *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, Document Version 20211203, RISC-V International, Dec. 2021.
- [3] Arm Ltd., *AMBA AXI and ACE Protocol Specification*, IHI 0022, 2021, ch. B “AMBA AXI4-Lite”.
- [4] *IEEE Standard for Test Access Port and Boundary-Scan Architecture*, IEEE Std 1149.1-2013, 2013.
- [5] Xilinx Inc., *UltraScale Architecture Libraries Guide*, UG974, 2025 (“BSCANE2”, “BUFGCE_DIV”).
- [6] Xilinx Inc., *ZCU104 Evaluation Board User Guide*, UG1267, v1.1, 2018.
- [7] RISC-V Software Collaboration, “riscv-tests,” <https://github.com/riscv-software-src/riscv-tests>, commit 2ebecad, Aug. 2026.
- [8] Xilinx Inc., *Vivado Design Suite User Guide: Logic Simulation*, UG900, 2025.
- [9] Xilinx Inc., *Vivado Design Suite User Guide: Programming and Debugging*, UG908, 2025 (hardware-manager JTAG Tcl commands).